10. File Organizations and Indexes | Physical Schema

Summary

Tables are stored in data files. A file can be thought of as a sequence of "Data Records"

There can be multiple tables in a file or there can be multiple files for a table.

Data Files may not have any order. Data Files can be ordered data records on some field of a data record. Normally ordering of records is done on the primary key. Unordered files are called "heap files" while the other one is called "ordered files".

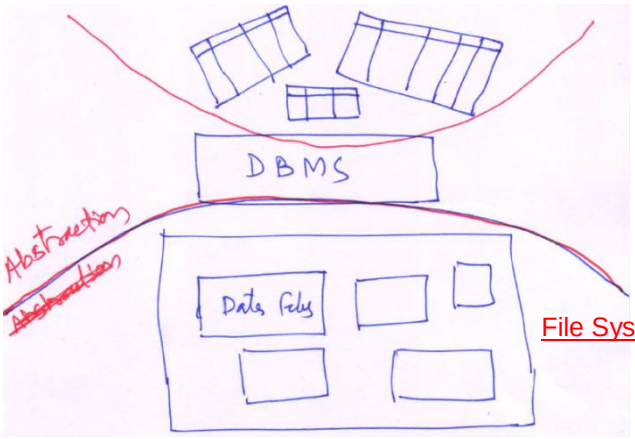
Performing manipulation operations on "data files" may turn out to be expensive. For instance, performing the insert operation is efficient in heap file but search operation is inefficient as it will have the complexity of order N. The Search operation on the ordered file will be better on the ordered file (as we can perform binary search) than the heap file.

Indexes are used for improving the search operations on data files. Indexes should also try their best to have acceptable performance on INSERT, UPDATE, and DELETE as well.

A relation is a logical representation of databases.

DBMS lets us manipulate databases in logical manner like relations, and systems like RDBMS provide relational abstraction over data in disk files.

Relation-tuples are stored in files on disks.



Files on the disk uses concepts of "Data Records", "Order of Data Records", "Disk-block" as a unit (a chunk of bytes) for reading and writing data into files.

A DBMS may use –

1. One disk files per relation, or
2. All relations are put into a single file; for example, SQLite, and MS-Access.
3. One relation many files.

Structure of data on disk is referred as *physical schema*.

Physical schema typically defines the organization of *data records* in file(s), and access paths for finding data records.

In this chapter, we attempt understanding various *file organizations* and *indexes* for efficient access of data records.

Data Files:

Logically, a *data file* can be thought of collection of *data records*. For storing relations, a record typically represents a *tuple*.

While studying file organizations, we use a concept of *disk block*. It is a smallest storage unit for manipulating data in disk files. A disk block – can be of different sizes on different systems - 4KB, 8KB, 16KB or so. 8KB should a standard on a modern DBMS.

A data file can be chained (or indexed) collection of blocks.

DBMS typically takes services of Operating system for manipulating disk files. Read/write operations are performed in terms of disk blocks.

A block may store multiple records; though rare, but it is also possible to have records that do not fit in a block, and span to multiple blocks.

Records in a file can be of two types –

1. "fixed length record" and
2. "variable length records"

These terms mean what they pronounce for.

If storage required for a record is fixed for all records in a data file, then it is fixed length record file; otherwise it is a variable length record file.

Length of a record in a file can be variable due to one and more of following reasons-

- If file stores tuples from more than one relations.
- Length of one or more fields is variable.
- A record having multi value fields.

Files with fixed length records are easier to implement and manage. However, fix length variable records are less practical and variable length record files are common in practice.

For implementing variable length record files, there are different strategies, like

1. Having record header containing information of length for each field, or
2. We can have field separators, or so.

We do not get into further details on this, here.

Motivating example:

Suppose we have a data file "students.dat" storing students relation, and would want to execute SQL query expression.

```
SELECT STUDID, NAME, CPI FROM STUDENT WHERE CPI > 7.5;
```

How do you think the students' data records file is to be processed for computing results for this query?

Operations on data files (physical operations)

Relational operations defined on relations are executed through some physical operations on the relevant data files.

Following are typical physical operations:

- Insert a record
- Find a record,
- Find a set of records meeting some criteria
- Modify a record
- Delete a record, or a set of records meeting some criteria

There are various techniques for performing these operations on files; however, the technique depends on "File Organization" used for recording data.

File Organizations

File Organizations is the way of organizing records in a file, that is, how records are represented, how attribute values are separated within a record, how records are sequenced, how records are searched, etc.

Every file organization defines some methods to locate a record (or a set of records) in a file, referred as *Access Methods*. There can be multiple access methods on file organization but an access method may require a particular file organization.

File organization dictates procedure of performing said operations on files. Each file organization makes certain operations efficient while others expensive.

Many alternative file organizations exist, following are commonly used file organizations-

- Heap (random order) files
- Sorted Files (on some attribute).
- Index Files

Heap (random order) files:

- Records have no order
- Insertions are very efficient; in this case, as new records are added at the end of file only.
- New records can also be placed at empty spaces anywhere in between of the file, created by deletion or so.
- Finding a record is expensive, as we can only perform sequential scan – in the order of N.

Sorted Files

- Records of file are sorted on some field(s), called as ordering field(s). Ordering field(s) is also called as search key. For example, you can store employee relation records sorted on ENO, or Department?
- Searching on ordering field is efficient as we can perform things like binary search – locate middle block, see if search key <value in ordering field of middle block, and so on.
  ==> Number of disk blocks needs to be processed is in the order of log2(N).
- Insertion in sorted file is expensive; can be performed as
  - Find appropriate location (block) in file where the record can be inserted (order needs to be maintained)
  - Once a neighborhood is found, it may encounter a few problems
    - One, the block containing the neighborhood has room where the new record can be inserted –if so, the new record is placed in that block
    - The block does not have room for new record – then see if next block has the room (and that still maintains order),
    - If still not possible, then an overflow block is added, and the new record is inserted there.
- Deletion is also expensive; done as following
  - Find record, record content is removed from the block; actual action will depend on how records are organized in a block.
  - One of the common strategies for space created by deleting a record is, keep it as blank and we do not shift the remaining records; such empty spaces are recycled using some strategy like "*free list*" maintenance or so.

- In a record modification operation, if we need to change values of ordering fields; then location of the record will change, in such as modification is performed in two steps-
    - Remove from current location
    - Insert at appropriate new location

"Cost" of an operation?

Disk I/O is considered as most significant component in performing database operations. Total number of disk blocks read and written measures the cost of performing an operation. A "query executing technique" that processes (reads and writes) larger number of disk block is considered more expensive than the one that processes lesser number of blocks.

Suppose a file has 10 lac records. Say (1) we have block-size of 4K. (2) a record uses 350bytes. As a result, we can store approximately 10 records in a block.
The file will be using one lac blocks.

Now for searching, if file is a "Heap File", how many blocks we need to process?

In worst case, we need to read all the blocks, i.e. one lac.

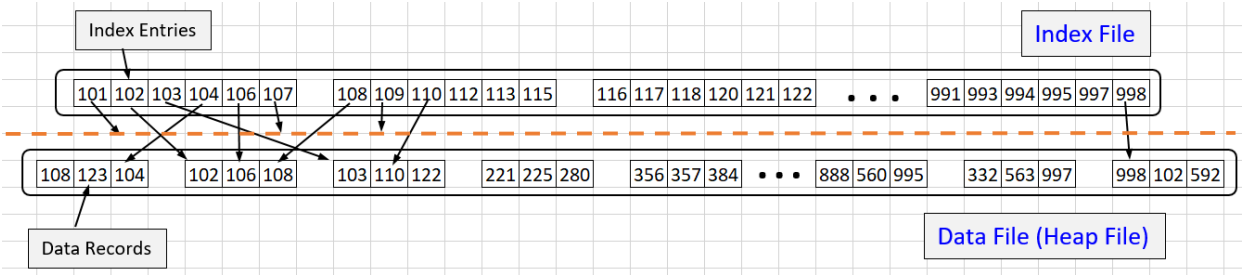
If file is ordered then we can perform binary search, then we need to process $\lceil log_2(100000) \rceil = 17$ blocks. Searching an "ordered file" is much efficient in comparison to a heap file. Right?

Indexes

Although sorted files do improve the search performance in logarithmic order, and have following issues -

1. Cost of "binary searching" could still be large in very large data files
2. A file can be ordered only on one attribute (or attribute set) only
3. Complexity of insert/delete/update is higher
4. Indexes further speed up searches for a subset of "records" based on values in certain fields; termed as "search key".

Figure below depicts a sample maps (index file).



An index is a file that maps a "search-key value" to data record address (typically address of block containing corresponding data record). It is an organized collection of "index records"; and an index record stores "search-key" and address of data record(s) with the search-key.

A "Search keys" for index is -

- Any subset of the attributes of a relation can be the search key for an index on the relation
- Search key may not be same as key of a relation.
- Typically, index has entries as following: `<search-key, block/record-address>`, or `<search-key, block/record-address list>`

Below are some types of entries in an index file records -

- `<Key,Address>` pais. For example, Student relation having index on SID
  `<101, adr-1>,`
  `<103, adr-2>,…`

- `<Key, Address-List>` pairs. For example, Employee relation having index on DNO
  `<1, <adr-1, adr-3, adr-7>>,`
  `<3, <adr-2, adr-5, adr-6>>,…`

- `<Composite-Key, Address>` pairs. For example, Employee relation having index on (DNO, Salary)
  `<<1, 30000>, adr-1>,`
  `<<3, 45000>, adr-6>,…`

Note that Index helps in reducing number of blocks to be processed for file operations; they are even at advantage over the ordered files.

Let us see some numbers to appreciate the index advantage:

Again, consider the same **data file with 10 lac records** that has an index. Size of index record will be much smaller than actual data record; let us say we can store 100 index records in a block; for ten lac index entries, we need 10000 blocks for index file. Index-records are always sorted; therefore, we can always perform binary search. In this case number of blocks to be read are $\lceil log_2(10000) \rceil = 14$. It was 17 blocks in case of sorted data file. Not much gain, though!

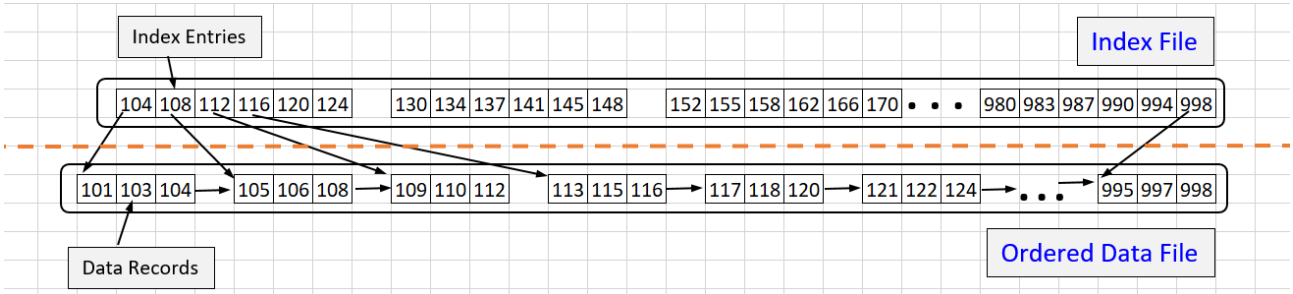
However, in this case we considered heap data file, where data records are not sorted. Therefore, the index file requires index entry for every data records. Such (that have index entry for every data record) are called as **Dense Index**).

If we have data file is also sorted (on the same search-key that of index), we do not store an index entry for every search-key value in the file. Since the data records are ordered, we can

instead store the address of either the minimum or maximum key value in each data block. Thus, there is only one index entry per data block. As a result, the number of index entries are significantly reduced and becomes equal to the number of data blocks in the data file.

Such an index is called a Sparse Index (that is, an index that does not have index entry for all data records).

Figure below depicts a "sparse index" on sorted data file.



Note that in the index above, we have index entry of highest search key in a block. This means, an address, in an index record, is where keys <= the key-value are stored. For example pointer at Key=104, refers to a block where data records with key <= 104 are stored. Similarly the address at 116 refers to a block where search key <=116 and > 112 are stored, and so forth.

Let us do some calculations here.

For the example above, if we have data file sorted, we shall have only 100,000 index entries to be stored (rather than 10 lac, why?) and **would require only 1000 blocks for index** (instead of 10000), resulting into the cost of $\lceil log_2(1000) \rceil = 10$ blocks.

Note the gains:

- one lac blocks – when we search on the data file (i.e. un-ordered file)
- 17 blocks – again on data file but when the file is ordered
- 14 blocks – when we search through index and have dense index when data file is not ordered
- 10 blocks – when we search through, have sparse index and data file is also ordered on same search key

This gain is because we are able to reduce the number of blocks to be processed.

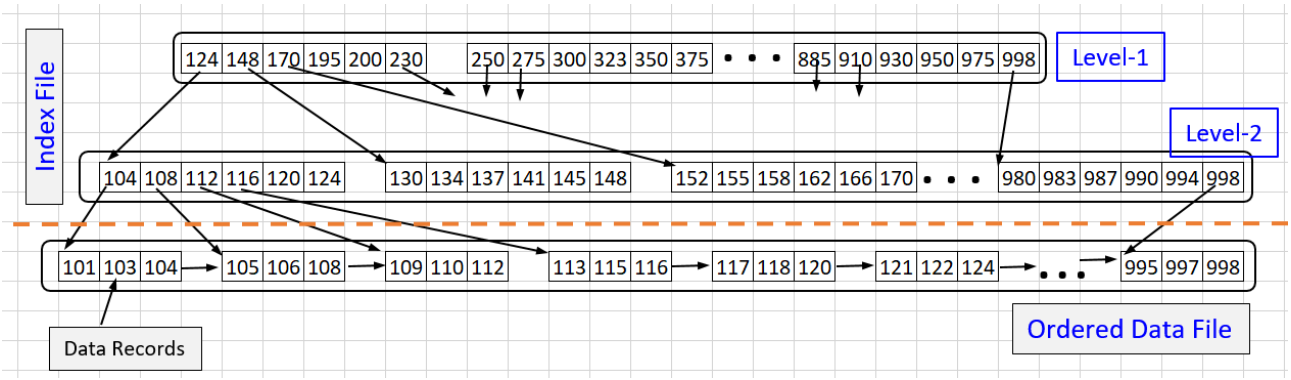
Actually, we can further reduce it by *multi-level indexes*.

Multi-level index

To understand the concept, let us continue on the same example.

Let us consider the case where data records are 10lacs; requiring 1 lac disk blocks. Data Records are sorted, therefore can have a sparse index. Our index would require 1 lac index records, and would use 1000 blocks (assuming 100 index records can be fitted into one block).

Suppose we create one higher-level sparse index that holds one entry for each index block. Let us call this new index as Level-1 index while original index as Level-2 index, as shown in figure below



At index Level-1, for storing 1000 index entries (of level-2), we require 1000/100 = 10 blocks.

Therefore, size of index at Level-1 is 10 blocks, while size at Level-2 is 1000 blocks. Let us say data record addresses are stored only at lowest level, which is Level-2 here.

Now, let us say, searching is performed as following-

Perform binary search at level-1, and locate block that contains address of block at level-2 that possibly contains address of data record. Number of blocks read at level-1 are $\lceil log_2(10) \rceil = 4$ blocks.

1. Read the corresponding block at level 2, and find out the address of the data record. The number of blocks processed at level-2 is one.

Total number of blocks read in this approach are FIVE.

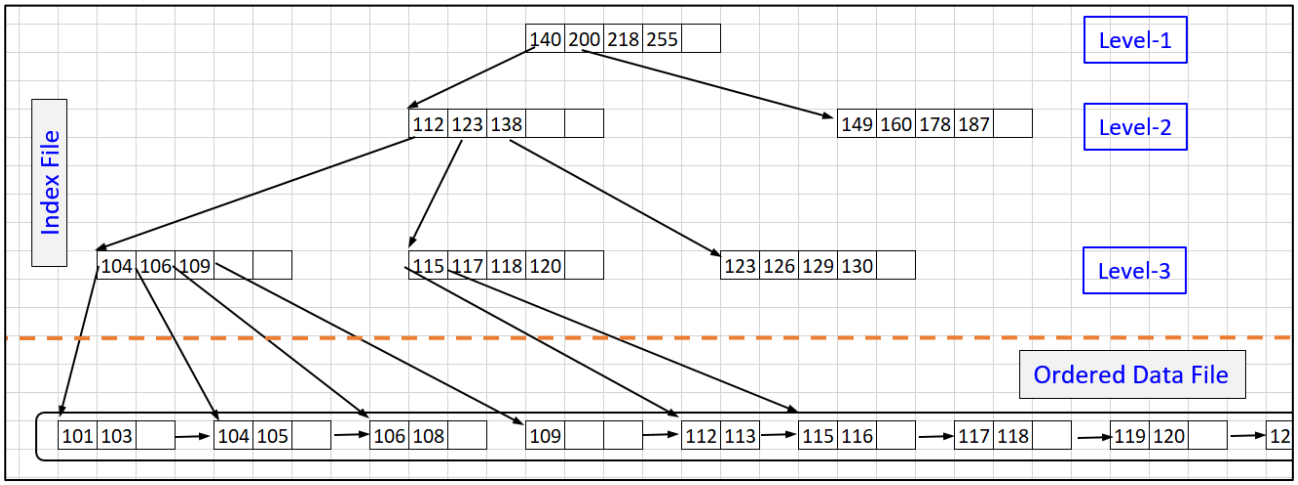
The Gain: 10 blocks to 5 blocks by creating 2-level index.

Now suppose we create index at one more level. Now referring them level-1, level-2, level-3. At level-3, there are 1000 blocks. At level-2, there are 10 blocks to stored addresses of 1000 blocks. At level-1, we require to store only 10 index record, and would use only one block.

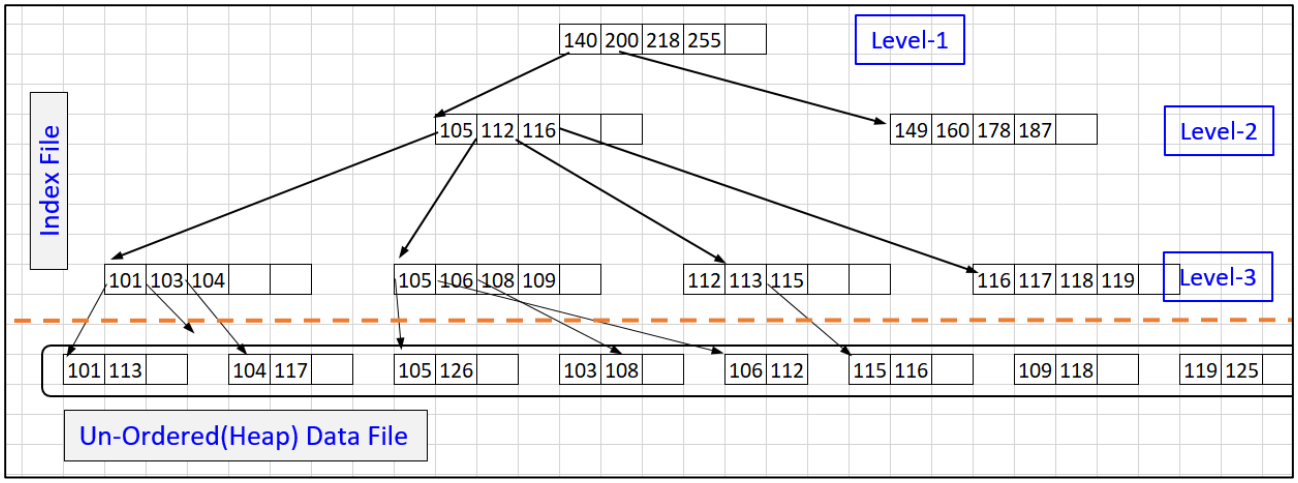
Total Cost here shall be: “1+1+1” = 3 blocks, PLUS 1 for reading data block.

Number of levels are typically determined such that index at top level (or root level) is small enough to be accommodated in single block. As a result, we just need to read one disk block at each level.

Thus, in general cost for locating a data record = number of levels + 1



A depiction of 3-level index (Sparse/Ordered File)



A depiction of 3-level index (Dense/Heap File)

Calculation Summary

Number of Records: 10 Lacs
Record Size: 350 B
Block Size: 4KB
Number of Records/Blocks: 10
Number of Blocks used by File: 100,000

1-Level Index:

Number of Index Records per block, say 100 entries/4KB block

Heap File (Un-Ordered)	Ordered File
File Search Cost: 100,000 blocks	Log2(100000) = 17 blocks
Index Search: Number of blocks used by "**10 lac index entries**" =10 lac/100= 10000 blocks Cost = log2(10000) = 14 blocks	Index Search: "Spars Index is possible" Only 100,000 index entries are required blocks used by index =100,000/100= 1000 blocks Cost = log2(1000) = 10 blocks
Total Cost=14 for Index Records + 1 Data Block=15	Total Cost=10 for Index Records + 1 Data Block=11

2-Level Index:

Heap File (Un-Ordered)	Ordered File
Level-2 Index Search: Number of blocks used by "**10000 index entries**" =10000/100=100 blocks Cost = log2(100) = 7 blocks	Level-2 Index search: Only 1000 index entries are required blocks used by index =1000/100= 10 blocks Cost = log2(10) = 4 block
Level-1 Index Search: 1 block	Level-1 Index Search: 1 block
Total Cost=7 (L2) + 1(L1) + 1 = 9 blocks	Total Cost=4 (L2) + 1(L1) + 1 =6 blocks

3-Level Index:

Heap File (Un-Ordered)	Ordered File
Level-3 Index Search: Number of blocks used by "**100 index entries**" =100/100=1 blocks Cost = 1 block	Level-3 Index search: Only 10 index entries are required blocks used by index =1 block Cost = 1 block
Total Cost=1(L3)+1(L2)+1(L1) + 1 = 4 blocks	Total Cost=1(L3)+1(L2)+1(L1) + 1 = 4 blocks

B+-tree Index

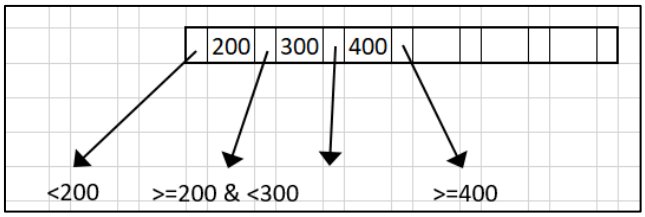
B+-tree is a higher-order search tree. Higher order means a node has more than two children nodes. Shown below is an example node of B-Tree.

Characteristics of B+-trees indexes:

- A node in a B-Tree is typically disk block having "fan-out" of "n", called degree of the tree. This means a node has "n" number of pointers (at-max) and n-1 search keys.
- Here is a depiction of structure of a node, where P_1 through P_n are pointers, and K_1 through K_{n-1} are key values.

P_1	K_1	P_2	…	P_{n-1}	K_{n-1}	P_n

- Key values in a node are ordered.
- For example, here is a node an internal node of a tree with n=7
- however, it has only 3 search keys and 4 pointers. A pointer in the left of a search key points to another node at lower level that has search keys < the key. Whereas the right hand side pointer points to a node at next level with search keys >= the key.

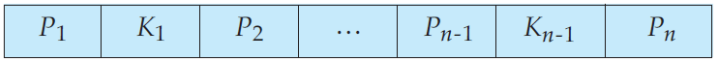


- Only leaf nodes store pointers (addresses) for data records. This enables increasing the number of pointers that can be stored in a node. And ultimately reducing the size of tree.
- Size of all nodes is same.
- Nodes may not be used for full capacity; some empty space is kept to accommodate efficient inserts. [Min number of pointers = $\lceil (N-1)/2 \rceil$ and max is about 66% to 75%].
- Intermediate (Non-leaf) nodes
  - Non leaf nodes contain pointers to its children nodes
  - A pointer P_i in a node points to record with search key >= K_{i-1} and < K_i
  - A search value appearing parent node will also appear in child node.
- Leaf nodes
  - Leaf nodes only contain pointers to data records.
  - Leaf node will contain pointer to all data records in data file, if it is dense index, which is usually the case is.
  - The pointer P_n (that is last pointer) of a leaf node point to next leaf node, and thus forming the linked list of leaf nodes.
  - A pointer P_i in a leaf node points to record with search key = Ki, if it is a dense index,
  - otherwise points to search-keys <= K_i and > K_{i-1}
  - **All leaf nodes combined will be in the order of search key.**
  - Leaf node may also be linked doubly; may have a pointer like P_0 that refers to previous node.

B+-tree is a multi-level index based on balanced binary search tree. It is a kind of balanced search tree adapted for database indexing and is a popular database indexing technique.

Characteristics of B^+-trees indexes:

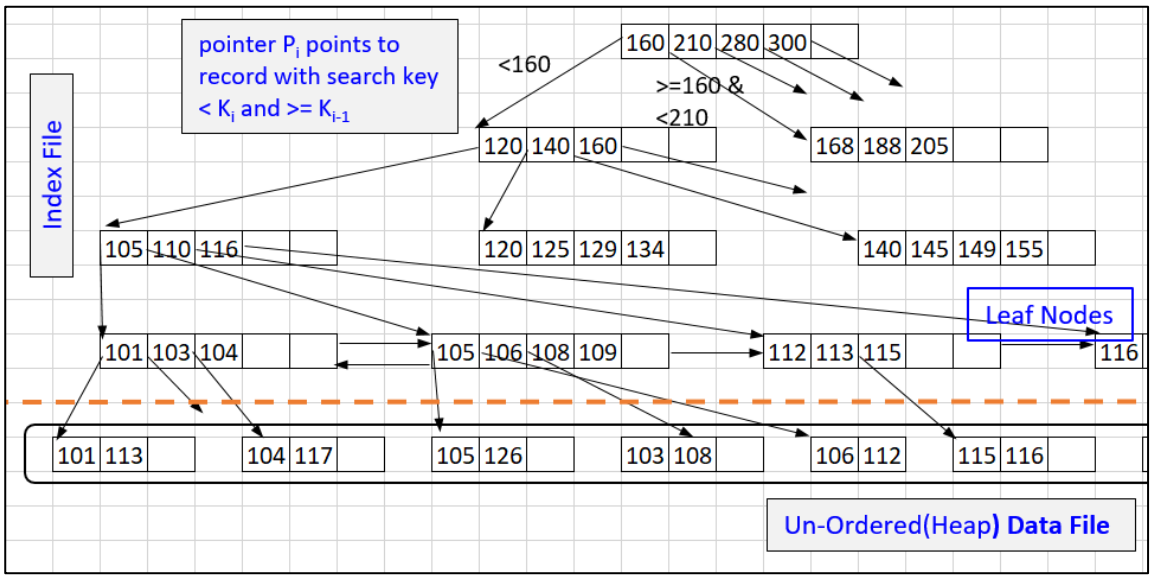
- Only leaf nodes store address for data records. This enables increasing the number of pointers that can be stored in a node. And ultimately reducing the size of tree.
- A node is typically disk block; below is a general B+-tree node with n index entries-



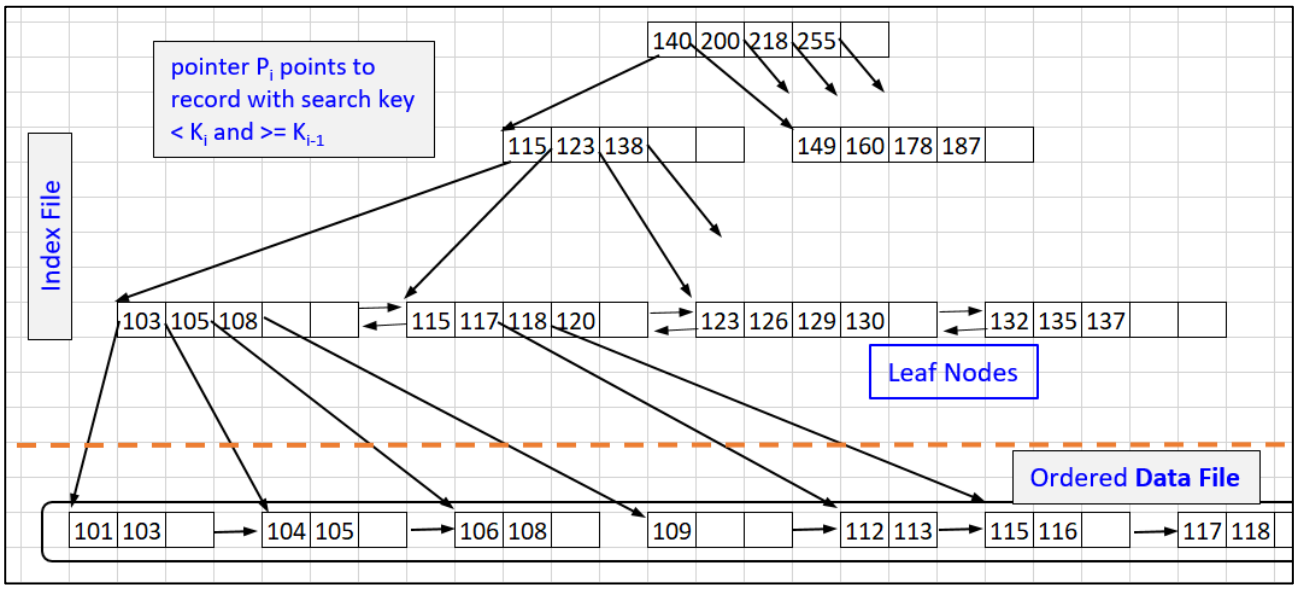
  where P_1 through P_n are pointers, and K_1 through K_{n-1} are key values.
- Key values in a node are ordered.
- Size of all nodes is same.
- Nodes may not be used for full capacity; some empty space is kept to accommodate efficient inserts. [Min number of pointers = $\lceil (N-1)/2 \rceil$ and max is about 66% to 75%].
- Leaf nodes
  - Leaf nodes only contain pointers to data records.
  - The pointer P_n (that is last pointer) of a leaf node point to next leaf node, and thus forming the linked list of leaf nodes.
  - A pointer P_i in a leaf node points to record with search key <= K_i and > K_{i-1}
  - All leaf nodes combined will be in the order of search key.
  - Leaf node may also be linked doubly; may have a pointer like P_0 that refers to previous node.
  - Leaf node will contain pointer to all data records in data file, if it is dense index, which is usually the case is.
- Non-leaf node
  - Non leaf nodes contain pointers to its children nodes
  - A pointer P_i in a node points to record with search key < K_i and >= K_{i-1}
  - A search value appearing parent node will also appear in child node.

Figure below should be capturing the conceptual layout of a B^+-tree index. Note that leaf nodes are referring *data records*.

Dense Index on Un-Ordered Data File



Sparse Index on Ordered Data File



Hopefully, you understand how we perform following operations on b-tree; almost identical algorithms can be applied on a disk-based B^+-tree of disk blocks containing index entries-

- Search: equality search, range search
- insert
- delete

Types of Indexes

- Primary and Secondary
- Unique and non-Unique
- Dense and sparse
- Clustered and non-clustered

Structures of indexes

- Tree based
- Hash based

There can be multiple indexes on a data file.

Clustered and non-clustered Index

- If records on disk file are sorted in index order then it is clustered index otherwise not.
- We can have data clustered only on one index.

Dense and Sparse Indexes

- If you have index entry for every search key-value in a file (relation), then it is dense index.
- "Clustered indexes" can only be sparse, where you do not store disk page address for each record. Advantage here is you have a smaller index and can load more portion of index in primary memory.

Primary Index

- An index based on attributes on which the data file is also physically sorted is referred to as a *primary index.* In most cases, the primary index is based on the primary key.

Secondary Index

Indexes other than primary indexes are secondary indexes, and

- Can be on any attributes
- Need not be unique
- However, a secondary index is always dense
- There can be multiple secondary indexes on a file of records to access records in multiple ways

Clustered vs. Un-clustered Indexes

- If the order of data records is the same as, or "close to"; order of data entries in the index, then the index is called a clustered index.
- A file can be clustered on at most one search key.

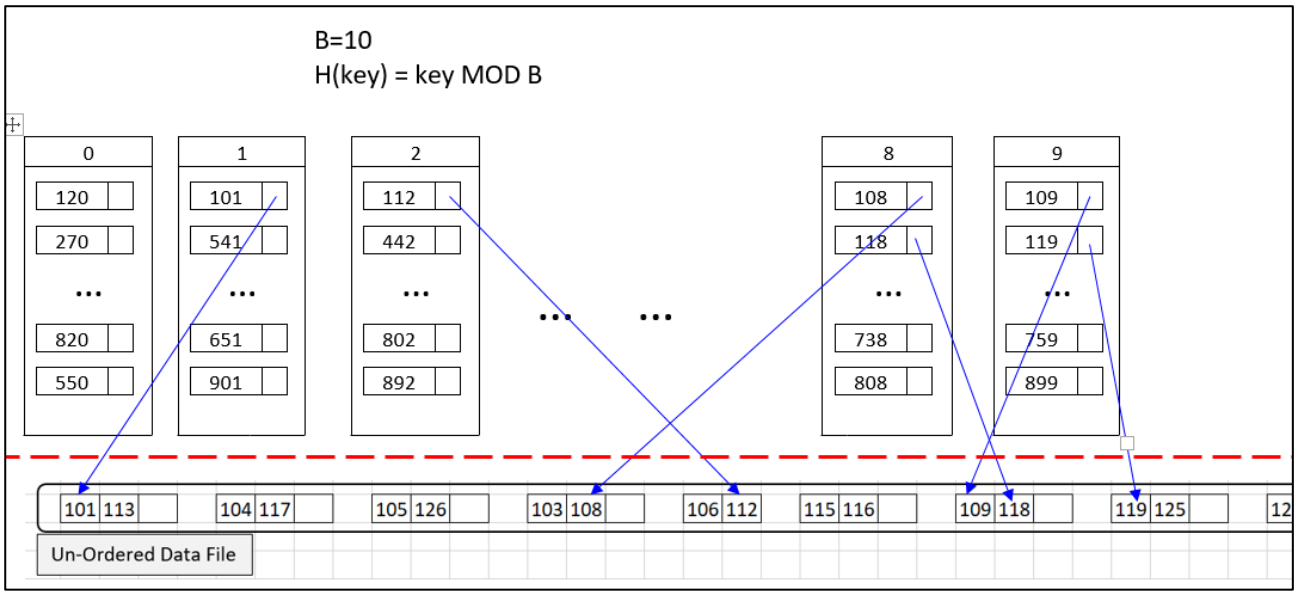
Hash based Indexes

As we know, hashing can be used for storing and searching. A search key maps to an "in-memory" "bucket". A bucket may itself store the data object or keep a pointer to the data object.

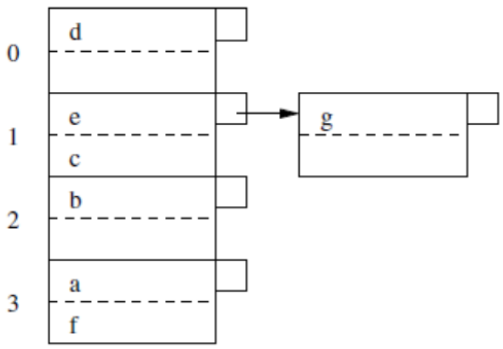
If we want to use this concept in databases, the hash table needs to be stored on disk. This is done by using disk blocks as buckets. We may refer to them as "**Persistent Hash Tables**"

We can use hash tables for data records themselves, or can create secondary index using "persistent hash tables", where "index entries" are stored in buckets. In that case, an index is a collection of buckets

A hash function **h** is used such that **h(k)**=bucket in which index entry for key k.



While "Hash Table" based indexes are traditionally considered better for "equality" searches, also referred to as "pointed queries". Range queries would turn out very inefficient through hash-based indexes.



"Overflow" chains are known problem with Hashing, it is applicable when hashing is used for indexing. Following are popular techniques to deal with this problem

- Linear Hash Tables
- Extensible Hash Tables

Maintenance of Indexes

Update on data file requires updating indexes as well

Below are some observations related to updating B+-tree based indexes-

- When we update data files (insert/update/delete), we need to updates all indexes (as said, there can be multiple indexes on a data file)
- To insert a record in data file, we also need to make appropriate index entry in B+-tree node; this new entry may require we splitting the node, and may further require balancing of tree, etc.
- While deleting a record from data file, we also need to remove corresponding index entry from B+-tree node; if the node becomes empty more than (let us say k/2), we require merging it with some other node,

Similar is the case with Hash based indexes

- Hash table needs to be updated as data in data file inserted/updated/deleted!

Indexes – Benefit/Cost

- Indexes consume extra storage space – sometimes may be as large as database itself
- Still indexes are used, but a tradeoff is used for deciding number of indexes or so
- If selection queries on an attribute are frequent, building an *index* on that attribute is desirable. In the absence of an index often query shall be answered by sorting the file on that attribute!
- Index Maintenance: needs to be in sync with the data files and that slows down update (INSERT/Modify/Delete) operations.
- Reduces “concurrency”
- If table size is small – you may not need index, and if table is huge and rarely updated we may have index on almost all attributes that are frequently used in retrieval queries

Indexing support in PostgreSQL

- PostgreSQL provides the CREATE INDEX command. Though not part of standard SQL!!
  Short form Syntax -
  `CREATE [UNIQUE] INDEX index-name ON table [USING method] (col-name)`

  Method can be `btree`, `hash`, and some more. The default method is `btree`.

- Postgres's hash method hardly has any advantage over btree for equality check
- In most RDBMS, indexing on Primary Key is automatically created. For example, in employee relation, we get index named "`employee_pkey`" automatically created.

Example

```
CREATE INDEX EMP_DNO_IDX ON EMPLOYEE(DNO);

CREATE INDEX EMP_DNO_IDX ON EMPLOYEE USING BTREE(DNO);
```

We also ALTER INDEX and DROP INDEX commands, as follows

```
DROP INDEX EMP_DNO_IDX;
```

We can cluster the database based on some index, as follows

```
CLUSTER employee USING emp_dno_idx;

CLUSTER employee USING employee_pkey;
```

Rebuilding indexes

- Indexes may go "Corrupted" or "Bloated", and then we require rebuilding indexes. For the same we have re-index command
- Syntax: `REINDEX { INDEX | TABLE | DATABASE | SYSTEM } name`